

```
1 import os
2 import pickle
3 import sys
4 import numpy as np
5 import model as nn
6 import copy
7 import scipy
8 import scipy.io
9 import argparse
10 from tqdm import tqdm
11 from utils import rotate_3d_coordinates
12 import matplotlib.pyplot as plt
13
14 def train(nnsgd, nnavg, R, Z, E_true, mb, hist,
15         augment):
16     rmses = []
17     for i in tqdm(range(1,100000)):
18
19         # learning rate schedule
20         if i > 0:      lr = 0.0000001
21         if i > 500:    lr = 0.00000025
22         if i > 2500:   lr = 0.0000005
23         if i > 12500:  lr = 0.000001
24         lr = lr*0.2
25
26         #sample minibatch indices
27         r = np.random.randint(0,len(R),[mb])
28         if augment:
29
30             R_aug = np.empty_like(R[r])
31
32             for j in range(mb):
33                 x_angle = np.random.uniform(-180, 180
34             )
35                 y_angle = np.random.uniform(-180, 180
36             )
37                 z_angle = np.random.uniform(-180, 180
```

```

35 )
36         #I realize that this now takes
        radians as argument, however, mathematically it
        shouldn't matter (even random dist for a sinusoidal
        fn)
37         R_aug[j] = rotate_3d_coordinates(R[r
        ][j], x_angle, y_angle, z_angle)
38         R_batch = R_aug
39     else:
40         R_batch = R[r]
41         E_pred = nnsgd.forward((R_batch, Z[r]))
42         loss = 0.5 * np.sum((E_pred - E_true[r]) ** 2
    ) # MSE loss
43
44         rmse = np.square(E_pred-E_true[r]).mean(axis=
    0)**.5
45         rmses.append(rmse)
46         nnsgd.backward((E_pred- E_true[r])) #TODO:
        add argument to backward
47         nnsgd.update(lr)
48         nnavg.average(nnsgd,(1/hist)/((1/hist)+i))
49         nnavg.nbiter = i
50
51         if i % 100 == 0:
52             print(f"RMSE: {sum(rmses[-100:])/100}
        kcal/mol")
53         pickle.dump(nnavg,open(f'nn-augment={
        augment}.pkl','wb'),pickle.HIGHEST_PROTOCOL)
54         np.save(f'training-rmses-augment={augment}.npy',
        np.array(rmses))
55         print("Done Training")
56
57
58 if __name__ == '__main__':
59     '''Setup'''
60     parser = argparse.ArgumentParser()
61     parser.add_argument('--seed', type=int, default=
        123, help='Random seed')

```

```

62     parser.add_argument('--augmentation', action='
store_true', help='whether to perform data
augmentations')
63     parser.add_argument('--train_fraction', type=
float, default=0.5, help='Fraction of training split
to use')
64     parser.add_argument('--mb', type=int, default=25
, help='Batch size')
65     parser.add_argument('--hist', type=float, default
=0.1, help='Fraction of the history to be remembered
for Exponential Moving Average')
66     args = parser.parse_args()
67
68     np.random.seed(args.seed)
69
70     '''Load data'''
71     if not os.path.exists('qm7.mat'): os.system('wget
http://www.quantum-machine.org/data/qm7.mat')
72     dataset = scipy.io.loadmat('qm7.mat')
73     split_idx = dataset['P'][1:].flatten() #leave
first split for testing
74     skip_every = int(1/args.train_fraction)
75     R = dataset['R'][split_idx][::skip_every]
76     Z = dataset['Z'][split_idx][::skip_every]
77     E_true = dataset['T'][0,split_idx][::skip_every]
78
79     '''Create neural network'''
80     #Define input and output layers
81     I, O = nn.Input((R, Z)), nn.Output(E_true)
82     nnsgd = nn.Sequential([
83         I,
84         nn.Linear(92, 400),
85         nn.Tanh(),
86         nn.Linear(400, 100),
87         nn.Tanh(),
88         nn.Linear(100, 1),
89         O
90     ])

```

```
91     nnavg = copy.deepcopy(nnsgd)
92
93     '''Train neural network'''
94     train(nnsgd, nnavg, R, Z, E_true, args.mb, args.
    hist, augment= True)
95
```